Análisis de algoritmos · 2026B

OPTIMIZADOR Y SELECCIONADOR DE TEMAS MUSICALES UTILIZANDO GRAFOS

12 de noviembre de 2026

Montalvo Hernández Raúl Octavio
Aguilar Rivera Victor Manuel

Problema

La reproducción aleatoria mezcla canciones con ritmos y géneros distintos y puede generar transiciones incongruentes.

Desarrollo de una herramienta que organice una lista de temas musicales congruente entre ellas a partir de un grafo con diferentes pesos, con el propósito principal de calcular y hacer una secuencia lineal óptima el cual logre poder crear una congruencia entre los diferentes temas musicales, utilizando un algoritmo de árbol de expansión mínima y un recorrido secuencial para poder.

Proceso

1. Entrada
2. Atributos
3. Pesos
4. Grafo
5. Kruskal
6. DFS
7. Resultados

Estructura del programa

NOMBRE	FUNCION
interfaz.py	Punto de entrada, filtros, construcción de escenas e interacción gráfica.
main.py	Autenticación de Spotify y consulta de características en ReccoBeats. Su nombre no lo convierte en el punto de entrada utilizado
clasificador.py	Validación de características, cálculo de distancia y enriquecimiento de datos musicales.
generos.py	Normalización de nombres y consulta de géneros del álbum en Deezer.
algortimo.py	Generación de pares y algoritmo de Kruskal. Se conserva el nombre real del archivo.
recorrido.py	Recorrido DFS de las conexiones seleccionadas.

Calculo distancias

CARACTERÍSTICA	CANCIÓN A	CANCIÓN B	DIFERENCIA
Género	Rock	Pop	G = 0.50
Tempo	100 BPM	120 BPM	T = 20/100 = 0.20
Energía	0.40	0.70	E = 0.30
Densidad	0.50	0.60	D = 0.10
Brillo	0.30	0.50	B = 0.20
Valencia	0.60	0.80	V = 0.20

Calculo distancias

$$d(A,B) = 30(0.50) + 20(0.20) + 20(0.30) + 10(0.10) + 10(0.20) + 10\,(0.20)$$

$$d(A,B) = 15 + 4 + 6 + 1 + 2 + 2$$

$$d(A,B) = 30$$

Kruskal

- Ordena las aristas por peso ascendente.
- Une raíces distintas y conserva la arista.
- Descarta aristas que formarían un ciclo.

```
def kruskal(canciones, candidatas=None):
    if candidatas is None:
        candidatas = conexiones(canciones)
    n = len(canciones)
    grupos = list(range(n))
    elegidas = []
    for conexion in sorted(candidatas, key=lambda c: c.peso)
        a = grupos[conexion.origen]
        b = grupos[conexion.destino]
        if a != b:
            elegidas.append(conexion)
            for i in range(n):
                if grupos[i] == b:
                    grupos[i] = a
    return elegidas
```

T(n): Kruskal

```
def kruskal(canciones, candidatas=None):
    if candidatas is None:                          # C1
        candidatas = conexiones(canciones)          # Se analiza al final

    n = len(canciones)                               # C2
    grupos = list(range(n))                          # C3 · n
    elegidas = []                                    # C4

    for conexion in sorted(                          # S(n): ordenar
        candidatas, key=lambda c: c.peso
    ):
        a = grupos[conexion.origen]                  #
        b = grupos[conexion.destino]                 # ┐ C5 por arista,
        if a != b:                                   # ┘ incluido control del for
            elegidas.append(conexion)                # C6 por unión

            for i in range(n):                       #
                if grupos[i] == b:                   # ┐ C7: cota por posición
                    grupos[i] = a                    # ┘

    return elegidas                                  # C8
```

Ordenar con sorted

$$m = \frac{n(n-1)}{2} \qquad S(n) = C_6 n^2 \log_2 n$$

$$T(n) = C_1 + C_2 n + S(n) + \sum_{j=1}^{\frac{n(n-1)}{2}} C_3 + \sum_{j=1}^{n-1} \left(C_4 + \sum_{i=1}^{n} C_5 \right)$$

$$T(n) = C_1 + C_2 n + S(n) + C_3 \frac{n(n-1)}{2} + (n-1)(C_4 + nC_5)$$

$$T(n) = C_1 + C_2 n + C_6 n^2 \log_2 n + \frac{C_3}{2}(n^2 - n) + C_4(n-1) + C_5(n^2 - n)$$

$$O(n^2 \log n)$$

Recorrido con busqueda de profundidad

Recorrido

Construye listas de adyacencia del bosque.
Usa una pila y marca los vértices visitados.
Devuelve un grupo por cada componente.

Alcance

La interfaz concatena los grupos.
Kruskal minimiza el peso total del árbol.
DFS no garantiza la mejor secuencia musical.

T(n): ordenar

```python
def recorrido_profundidad(canciones, conexiones):
    n = len(canciones)
    vecinos = [[] for _ in canciones]
    for c in conexiones:
        vecinos[c.origen].append(c.destino)
        vecinos[c.destino].append(c.origen)
    visitados = [False] * n
    def visitar(actual, grupo):
        visitados[actual] = True
        grupo.append(actual)
        for vecino in vecinos[actual]:
            if not visitados[vecino]:
                visitar(vecino, grupo)
    grupos = []
    for inicio in range(n):
        if not visitados[inicio]:
            grupo = []
            visitar(inicio, grupo)
            grupos.append(grupo)
    return grupos
```

$$n : canciones, m : conexiones, g : componentes.$$
$$m = n - g$$

$$T(n) = C_1 + C_2 n + \sum_{j=1}^{m} C_3 + \sum_{i=1}^{n} C_4 + \sum_{j=1}^{2m} C_5 + \sum_{i=1}^{n} C_6 + \sum_{k=1}^{g} C_7$$

$$T(n) = C_1 + C_2 n + C_3 m + C_4 n + 2C_5 m + C_6 n + C_7 g$$

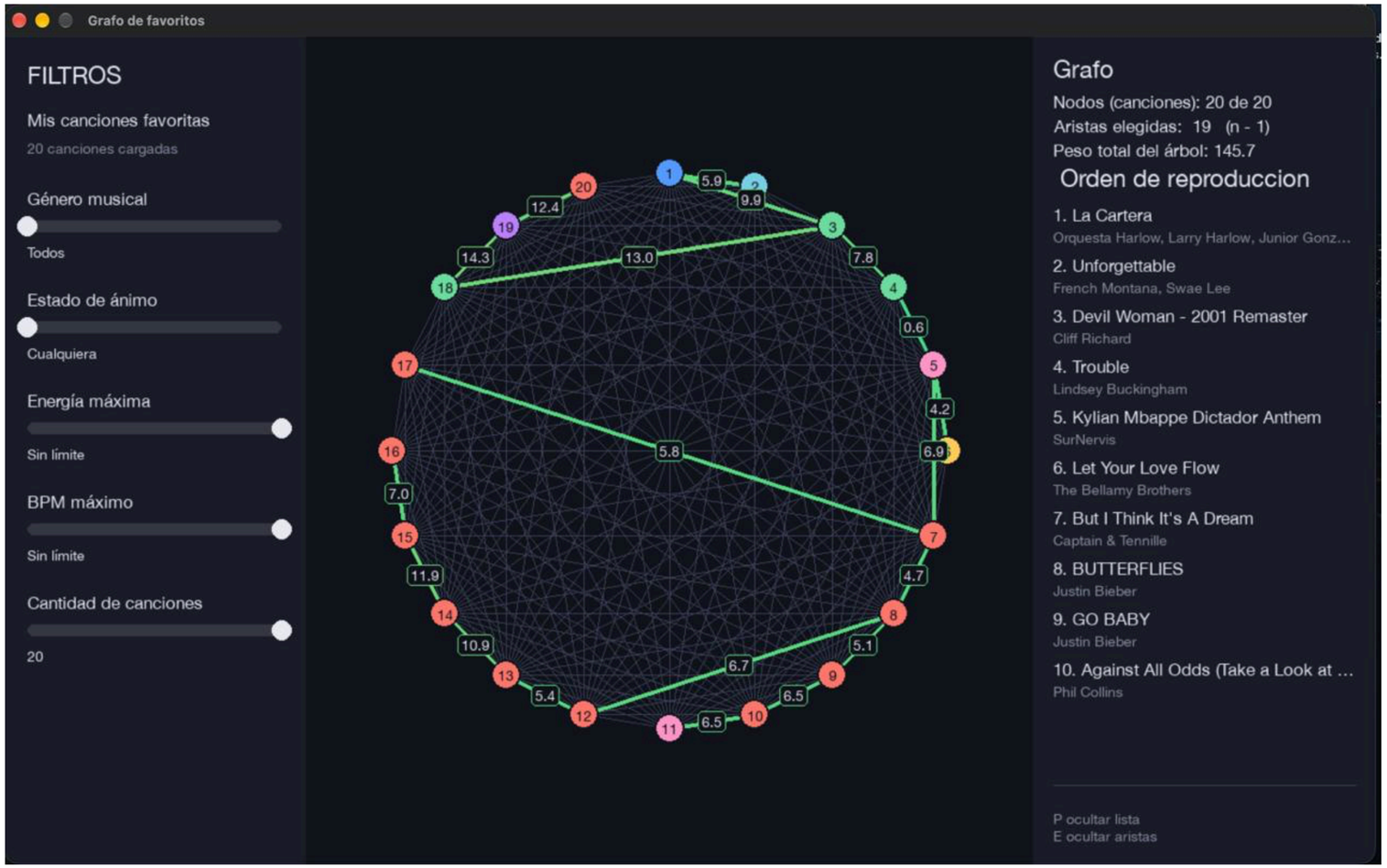
$$T(n) = C_1 + (C_2 + C_4 + C_6)n + (C_3 + 2C_5)m + C_7 g$$

$$T(n) = C_1 + (C_2 + C_4 + C_6)n + (C_3 + 2C_5)(n - g) + C_7 g$$

$$T(n) = C_1 + (C_2 + C_3 + C_4 + 2C_5 + C_6 + C_7)n$$

$$T(n) = O(n)$$

Resultado 1


Grafo de favoritos
FILTROS
Mis canciones favoritas
20 canciones cargadas
Género musical
Todos
Estado de ánimo
Cualquiera
Energía máxima
Sin límite
BPM máximo
Sin límite
Cantidad de canciones
20
Grafo
Nodos (canciones): 20 de 20
Aristas elegidas: 19 (n - 1)
Peso total del árbol: 145.7
Orden de reproduccion
1. La Cartera
Orquesta Harlow, Larry Harlow, Junior Gonz...
2. Unforgettable
French Montana, Swae Lee
3. Devil Woman - 2001 Remaster
Cliff Richard
4. Trouble
Lindsey Buckingham
5. Kylian Mbappe Dictador Anthem
SurNervis
6. Let Your Love Flow
The Bellamy Brothers
7. But I Think It's A Dream
Captain & Tennille
8. BUTTERFLIES
Justin Bieber
9. GO BABY
Justin Bieber
10. Against All Odds (Take a Look at ...
Phil Collins
P ocultar lista
E ocultar aristas

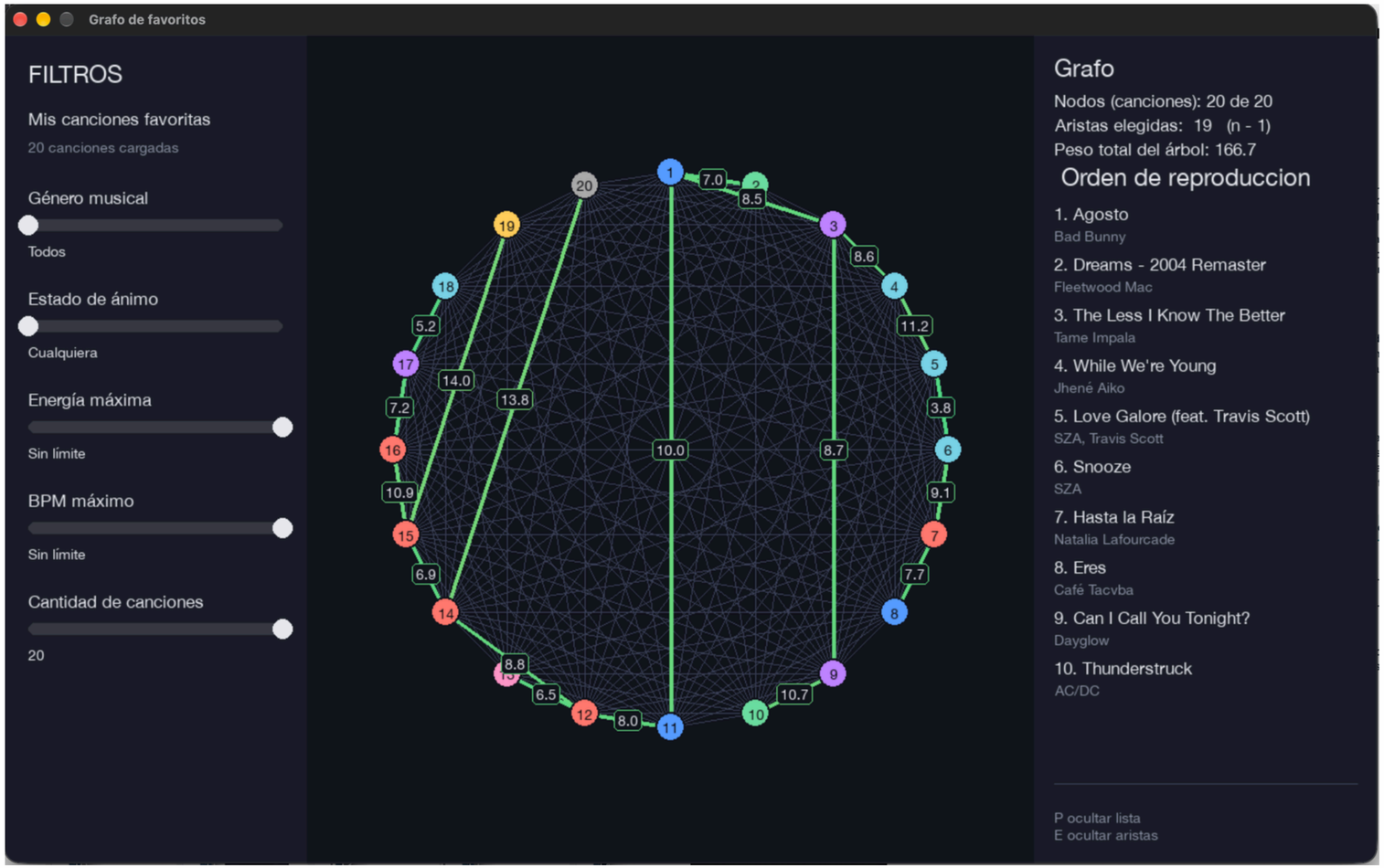
Resultado 1

El árbol obtuvo un peso de 145.7, con una distancia promedio de 7.67 por conexión.

La conexión entre los nodos 4 y 5 pesa 0.6, lo que indica una gran semejanza según las características comparadas.

Otras conexiones, como la de 18 y 19, con peso 14.3, permiten integrar canciones más diferentes y Kruskal combina conexiones de baja distancia con otras más costosas para mantener unido todo el conjunto. No exige que todas las canciones sean igualmente similares.

Resultado 2


Grafo de favoritos
FILTROS
Mis canciones favoritas
20 canciones cargadas
Género musical
Todos
Estado de ánimo
Cualquiera
Energía máxima
Sin límite
BPM máximo
Sin límite
Cantidad de canciones
20
Grafo
Nodos (canciones): 20 de 20
Aristas elegidas: 19 (n - 1)
Peso total del árbol: 166.7
Orden de reproduccion
1. Agosto
Bad Bunny
2. Dreams - 2004 Remaster
Fleetwood Mac
3. The Less I Know The Better
Tame Impala
4. While We're Young
Jhené Aiko
5. Love Galore (feat. Travis Scott)
SZA, Travis Scott
6. Snooze
SZA
7. Hasta la Raíz
Natalia Lafourcade
8. Eres
Café Tacvba
9. Can I Call You Tonight?
Dayglow
10. Thunderstruck
AC/DC
P ocultar lista
E ocultar aristas

Resultado 2

Con la misma cantidad de canciones y los mismos filtros, el peso aumentó a 166.7: un 14.4 % más que en la primera entrada. La distancia promedio por conexión fue 8.77.

El árbol presenta ramificaciones: el nodo 1 se conecta con 2, 11 y 20. Por ello, DFS debe cambiar de rama para visitar todas las canciones, y algunos pasos consecutivos de la lista no siguen una arista del árbol y Kruskal minimiza el peso del árbol, pero DFS no garantiza minimizar las distancias entre canciones consecutivas. El mayor peso indica mayor distancia acumulada según el modelo, no necesariamente peor calidad musical.

Conclusiones

Kruskal obtiene el árbol de menor peso para las distancias definidas. Sin embargo, el orden generado por el recorrido minimo, no garantiza las mejores transiciones entre canciones.

Se pueden ajustar los pesos y la matriz de géneros, mejorar la cobertura de datos y comparar el recorrido con otros métodos y con la reproducción aleatoria.

Cumplió el modelado mediante grafos, la construcción del árbol mínimo y la visualización del orden musical en las dos entradas probadas.

Referencias

1. Python Software Foundation. (s. f.). Sorting techniques. Python documentation. https://docs.python.org/3/howto/sorting.html
2. Python Software Foundation. (s. f.). itertools — Functions creating iterators for efficient looping. Python documentation. https://docs.python.org/3/library/itertools.html
3. Spotipy. (s. f.). Welcome to Spotipy!. https://spotipy.readthedocs.io/en/2.25.1/
4. Spotify. (s. f.). Get user’s saved tracks. Spotify for Developers. https://developer.spotify.com/documentation/web-api/reference/get-users-saved-tracks
5. ReccoBeats. (s. f.). Introduction. ReccoBeats documentation. https://reccobeats.com/docs/documentation/introduction
6. Pygame. (s. f.). Pygame documentation. https://www.pygame.org/docs/

Referencias

1. Requests. (s. f.). Requests: HTTP for humans. https://requests.readthedocs.io/en/latest/
2. FFmpeg. (s. f.). FFmpeg filters documentation. https://ffmpeg.org/ffmpeg-filters.html
3. MetaBrainz Foundation. (s. f.). MusicBrainz API. https://musicbrainz.org/doc/MusicBrainz_API
4. AcousticBrainz. (s. f.). Web API. https://acousticbrainz.readthedocs.io/api.html